

Task Manager JWT

Documentación Técnica Completa

Sistema fullstack de gestión de tareas
con autenticación JWT

PostgreSQL

Express.js

React

Node.js

[illegible]

[illegible]

1. Usuario hace una acción en React (p.ej. "crear tarea")
2. Axios adjunta el accessToken en el header Authorization: Bearer
3. Express verifica el token en authMiddleware !' popula req.user
4. Si el token es válido !' el controller ejecuta la lógica de negocio
5. Controller consulta o escribe en PostgreSQL via Prisma ORM
6. La respuesta JSON regresa al cliente
7. Redux Toolkit actualiza el estado global (slice de tareas)
8. React re-renderiza el componente afectado

1. Usuario hace una acción en React (p.ej. "crear tarea")
2. Axios adjunta el accessToken en el header Authorization: Bearer
3. Express verifica el token en authMiddleware !' popula req.user
4. Si el token es válido !' el controller ejecuta la lógica de negocio
5. Controller consulta o escribe en PostgreSQL via Prisma ORM
6. La respuesta JSON regresa al cliente
7. Redux Toolkit actualiza el estado global (slice de tareas)
8. React re-renderiza el componente afectado

[illegible]


```

task-manager-jwt/
% % % backend/
%   % % % server.js           !• Punto de entrada del servidor
%   % % % .env / .env.example
%   % % % package.json
%   % % % prisma/
%   %   % % schema.prisma      !• Modelos de base de datos
%   %   % % migrations/       !• Historial de migraciones
%   % % % src/
%       % % % config/
%       %   % % db.js          !• Instancia única de Prisma
%       % % % controllers/
%       %   % % % authController.js !• register, login, refresh, logout
%       %   % % % taskController.js !• CRUD tareas + exportCSV
%       %   % % % adminController.js !• getUsers, getUserTasks, updateRole
%       % % % middleware/
%       %   % % % authMiddleware.js !• Verificación JWT
%       %   % % % roleMiddleware.js !• Control de roles (admin/user)
%       %   % % % errorHandler.js  !• Manejo central de errores
%       % % % routes/
%       %   % % % authRoutes.js
%       %   % % % taskRoutes.js
%       %   % % % adminRoutes.js
%       % % % schemas/
%       %   % % % authSchemas.js   !• Zod: register, login
%       %   % % % taskSchemas.js   !• Zod: createTask, updateTask
%       % % % utils/
%       %   % % % jwt.js            !• Generación/verificación tokens
%       %   % % % csvExporter.js    !• Generación de CSV
%
% % % frontend/
%   % % % index.html              !• HTML raíz (Vite)
%   % % % vite.config.js
%   % % % src/
%       % % % main.jsx            !• Punto de entrada React
%       % % % App.jsx             !• Definición de rutas
%       % % % app/store.js        !• Redux store
%       % % % features/
%       %   % % % auth/authSlice.js !• Estado + thunks de autenticación
%       %   % % % tasks/tasksSlice.js !• Estado + thunks de tareas
%       % % % components/
%       %   % % % PrivateRoute.jsx  !• Protección de rutas
%       %   % % % TaskCard.jsx
%       %   % % % TaskForm.jsx
%       % % % pages/
%       %   % % % LoginPage.jsx
%       %   % % % RegisterPage.jsx
%       %   % % % DashboardPage.jsx
%       %   % % % AdminPage.jsx
%       % % % utils/
%       %   % % % axiosConfig.js    !• Axios + interceptores JWT

```



```
model User {
  id          Int          @id @default(autoincrement())
  email       String       @unique
  password    String       // Hash bcrypt, nunca texto plano
  name        String
  role        Role         @default(USER)
  refreshToken String?     // Último refreshToken válido emitido
  createdAt   DateTime     @default(now())
  tasks       Task[]       // Relación 1:N con Task
}

model Task {
  id          Int          @id @default(autoincrement())
  title       String
  description  String?
  status      TaskStatus  @default(PENDING)
  priority    Priority     @default(MEDIUM)
  dueDate     DateTime?
  userId      Int          // FK !' User.id
  user        User        @relation(fields: [userId], references: [id])
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
}

enum Role      { USER ADMIN }
enum TaskStatus { PENDING IN_PROGRESS COMPLETED }
enum Priority   { LOW MEDIUM HIGH }
```

[illegible]


```
# Crear y aplicar una migración nueva  
npx prisma migrate dev --name nombre_migracion
```

```
# Aplicar migraciones en producción  
npx prisma migrate deploy
```

```
# Insertar datos de prueba  
npx prisma db seed
```

```
# Abrir Prisma Studio (interfaz visual)  
npx prisma studio
```

```
# Resetear la BD completa (solo desarrollo)  
npx prisma migrate reset
```


6.1 Punto de Entrada: server.js

El archivo raíz del backend configura todos los middlewares globales y monta las rutas.

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');

const app = express();

// Seguridad: Helmet agrega headers HTTP protectores
app.use(helmet());

// CORS: solo acepta peticiones del frontend definido en CLIENT_URL
app.use(cors({ origin: process.env.CLIENT_URL, credentials: true }));
app.use(express.json());
app.use(cookieParser());

// Rate limiting: máximo 20 peticiones cada 15 min en /api/auth
const authLimiter = rateLimit({ windowMs: 15 * 60 * 1000, max: 20 });
app.use('/api/auth', authLimiter);

app.use('/api/auth', authRoutes);
app.use('/api/tasks', taskRoutes);
app.use('/api/admin', adminRoutes);

app.use(errorHandler); // siempre al final
```

6.2 Utilidades JWT (src/utills/jwt.js)

```
// Access token: vida corta (15 min) – viaja en el header Authorization
const generateAccessToken = (payload) =>
  jwt.sign(payload, process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_ACCESS_EXPIRES_IN || '15m',
  });

// Refresh token: vida larga (7 días) – viaja en cookie HTTP-Only
const generateRefreshToken = (payload) =>
  jwt.sign(payload, process.env.JWT_REFRESH_SECRET, {
    expiresIn: process.env.JWT_REFRESH_EXPIRES_IN || '7d',
  });

// Payload almacenado: { id, email, role }
```

Se usan dos secretos distintos (JWT_SECRET y JWT_REFRESH_SECRET) para que un atacante que comprometa uno no pueda forjar el otro tipo de token.

6.3 Schemas de Validación Zod


```
// authSchemas.js
const registerSchema = z.object({
  name:      z.string().min(2).max(100),
  email:     z.string().email(),
  password:  z.string().min(8).max(100),
});

// taskSchemas.js
const createTaskSchema = z.object({
  title:      z.string().min(1).max(200),
  description: z.string().max(1000).optional(),
  status:     z.enum(['PENDING', 'IN_PROGRESS', 'COMPLETED']).optional(),
  priority:   z.enum(['LOW', 'MEDIUM', 'HIGH']).optional(),
  dueDate:    z.string().optional().nullable(),
});
const updateTaskSchema = createTaskSchema.partial(); // todos los campos opcionales
```

6.4 Middlewares

authMiddleware.js — Verificación de JWT

```
const authenticate = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer '))
    return res.status(401).json({ message: 'No token provided' });

  const token = authHeader.split(' ')[1];
  try {
    req.user = verifyAccessToken(token); // adjunta {id, email, role}
    next();
  } catch {
    res.status(401).json({ message: 'Invalid or expired token' });
  }
};
```

roleMiddleware.js — Control de roles

```
// Higher-order function: acepta uno o más roles y devuelve un middleware
const requireRole = (...roles) => (req, res, next) => {
  if (!req.user || !roles.includes(req.user.role))
    return res.status(403).json({ message: 'Forbidden: insufficient permissions' });
  next();
};

// Uso en rutas:
router.get('/users', authenticate, requireRole('ADMIN'), getUsers);
```

6.5 Controlador de Autenticación

register

- Valida el body con `registerSchema.parse()` — falla con 400 si hay errores
 - Verifica que el email no esté registrado — devuelve 409 si ya existe

- Hashea la contraseña con `bcrypt.hash(password, 10)`
- Crea el usuario en PostgreSQL y devuelve 201 con datos básicos (sin contraseña)

login

- Valida con `loginSchema.parse()`
 - Busca el usuario por email y compara la contraseña con `bcrypt.compare()`
 - Genera `accessToken` + `refreshToken` con los datos del usuario (id, email, role)
 - Guarda el `refreshToken` en la columna `refreshToken` del usuario (para invalidación)
 - Envía el `refreshToken` como cookie HTTP-Only (inaccesible desde JavaScript)
 - Devuelve el `accessToken` + datos del usuario en el body JSON

refresh

- Lee el `refreshToken` de la cookie HTTP-Only
 - Verifica la firma del token con `verifyRefreshToken()`
 - Doble verificación: compara con el token guardado en BD — invalida tokens robados post-logout
 - Genera y devuelve un nuevo `accessToken`

logout

- Lee el `refreshToken` de la cookie
 - Pone `refreshToken: null` en la BD — invalida el token permanentemente
 - Limpia la cookie del cliente con `res.clearCookie()`

6.6 Controlador de Tareas

getTasks — Listar con filtros y paginación

```
const { status, priority, search, page = '1', limit = '10' } = req.query;
const where = { userId: req.user.id }; // solo tareas del usuario autenticado

if (status) where.status = status;
if (priority) where.priority = priority;
if (search) {
  where.OR = [
    { title: { contains: search, mode: 'insensitive' } },
    { description: { contains: search, mode: 'insensitive' } },
  ];
}

// Ejecuta count y findMany EN PARALELO para mayor eficiencia
const [tasks, total] = await Promise.all([
  prisma.task.findMany({ where, skip, take, orderBy: { createdAt: 'desc' } }),
  prisma.task.count({ where }),
]);
```

updateTask y deleteTask

Antes de modificar verifican que el recurso exista y que el usuario sea el dueño (`userId === req.user.id`) o tenga rol ADMIN. Esto previene la escalación horizontal de privilegios.

updateUserRole
en User Pack

Página 1

Cambia el rol entre USER y ADMIN — valida que el rol sea válido.
Devuelve todos los usuarios sin exponer contraseñas (select
value: string)

API Controlador de Administración

7.1 Redux Store (src/app/store.js)

Combina dos slices de estado.

```
{
  auth: { user, accessToken, loading, error },
  tasks: { items, total, page, loading, error }
}
```

El accessToken vive en memoria (Redux), nunca en localStorage. Esto lo protege de ataques XSS. Si el usuario recarga la página, el interceptor de Axios lo renueva automáticamente usando la cookie HTTP-Only.

7.2 Axios con Interceptores (src/utils/axiosConfig.js)

Interceptor de request

```
api.interceptors.request.use((config) => {
  const token = store.getState().auth.accessToken;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});
```

Interceptor de response — Renovación automática del token

```
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    const original = error.config;
    if (error.response?.status === 401 && !original._retry) {
      if (isRefreshing) {
        // Encolar petición hasta que termine el refresh
        return new Promise((resolve, reject) => {
          failedQueue.push({ resolve, reject })
        }).then((token) => {
          original.headers.Authorization = `Bearer ${token}`;
          return api(original);
        });
      }
      original._retry = true;
      isRefreshing = true;
      try {
        const res = await axios.post('/auth/refresh', {}, { withCredentials: true });
        store.dispatch(setAccessToken(res.data.accessToken));
        processQueue(null, res.data.accessToken);
        return api(original); // reintentar petición original
      } catch (err) {
        store.dispatch(logout()); // forzar re-login
      }
    }
  }
);
```

El resultado: el usuario nunca nota que su token expiró — la petición original se reintenta de forma transparente.


```
deleteTask(id)
updateTask(id, data))
```

DELETE /tasks/:id
POST /tasks/logout !' limpia el estado de autenticación

Actualizaciones del estado sin re-fetch: createTask agrega al inicio

índice, deleteTask filtra el array — el usuario ve los cambios inmed

7.5 PrivateRoute (src/components/PrivateRoute.jsx)

```
export default function PrivateRoute({ requiredRole }) {
  const { user, accessToken } = useSelector((state) => state.auth);

  if (!accessToken) return <Navigate to="/login" />;
  if (requiredRole && user?.role !== requiredRole) return <Navigate to="/dashboard" />;
  return <Outlet />;
}
```

Protege en dos niveles: sin sesión redirige al login, sin rol suficiente redirige al dashboard.

7.6 Enrutamiento (App.jsx)

```
<Routes>
  <Route path="/login" element={<LoginPage />} />
  <Route path="/register" element={<RegisterPage />} />
  <Route element={<PrivateRoute />>
    <Route path="/dashboard" element={<DashboardPage />} />
  </Route>
  <Route element={<PrivateRoute requiredRole="ADMIN" />>
    <Route path="/admin" element={<AdminPage />} />
  </Route>
  <Route path="*" element={<Navigate to="/login" replace />} />
</Routes>
```


Usuario	React Frontend	Express Backend	PostgreSQL
%	%	%	%
% % formulario	% %%	%	%
%	% % POST /auth/register	% %	%
%	%	% % INSERT user	% % % % % % % %
%	% % 201 { id, name, email }	% % % % % % % % % % % % % %	%
% % login	% % % % % %	%	%
% % credenciales	% %	%	%
%	% % POST /auth/login	% %	%
%	%	% % SELECT + compare	% %
%	%	% % UPDATE refreshToken	% %
%	% % { accessToken, user }	+ Cookie	% % % % % % % %
% % Dashboard	% % %	%	%

```

Frontend (Axios)                                Express Backend
%                                                    %
% % GET /tasks (token expirado)% % %
% % 401 Unauthorized % % % % % % % % % %
%                                                    %
% % POST /auth/refresh (cookie)% % %
% % { accessToken nuevo } % % % % % % %
%                                                    %
% % GET /tasks (token nuevo) % % % %
% % 200 OK + tareas % % % % % % % % % % % %

```


[illegible]

Backend — backend/.env

```
# Base de datos
DATABASE_URL="postgresql://usuario:password@localhost:5432/taskmanagerdb"

# JWT – usar strings de 64+ caracteres aleatorios
JWT_SECRET="string_muy_largo_y_aleatorio_para_access_tokens"
JWT_REFRESH_SECRET="otro_string_completamente_distinto_para_refresh"
JWT_ACCESS_EXPIRES_IN="15m"
JWT_REFRESH_EXPIRES_IN="7d"

# Servidor
PORT=3000
NODE_ENV=development  # cambiar a "production" en deploy

# CORS
CLIENT_URL="http://localhost:5173"

# Email (opcional)
EMAIL_HOST="smtp.gmail.com"
EMAIL_PORT=587
EMAIL_USER="tu@gmail.com"
EMAIL_PASS="tu_app_password"
```

Frontend — frontend/.env

```
VITE_API_URL="http://localhost:3000/api"
```


Requisitos previos

- Node.js v18 o superior
- PostgreSQL v14 o superior corriendo localmente
 - npm v9 o superior

Paso 1 — Clonar el repositorio

```
git clone https://github.com/tu-usuario/task-manager-jwt.git
cd task-manager-jwt
```

Paso 2 — Configurar el backend

```
cd backend
npm install
cp .env.example .env
# Editar .env con tus credenciales de PostgreSQL y secretos JWT
```

Paso 3 — Base de datos y migraciones

```
# Crear la base de datos en PostgreSQL
psql -U postgres -c "CREATE DATABASE taskmanagerdb;"

# Aplicar las migraciones con Prisma
npx prisma migrate dev

# (Opcional) Cargar datos de prueba
npx prisma db seed
```

Paso 4 — Iniciar el backend

```
npm run dev
# Servidor corriendo en http://localhost:3000
```

Paso 5 — Configurar e iniciar el frontend

```
cd ../frontend
npm install
cp .env.example .env
npm run dev
# Frontend en http://localhost:5173
```

Paso 6 — Build para producción

```
cd frontend
npm run build
# Los archivos estáticos quedan en frontend/dist/
```


[illegible]

Configuración Nginx

```
# /etc/nginx/sites-available/task-manager-jwt
server {
    listen 80;
    server_name 192.168.233.10;

    # Servir el build de React (Vite)
    root /home/emarin/projects/task-manager-jwt/frontend/dist;
    index index.html;

    # React Router: SPA fallback !' todas las rutas !' index.html
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Proxy al backend Express en puerto 3000
    location /api/ {
        proxy_pass          http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header    Host      $host;
        proxy_set_header    X-Real-IP $remote_addr;
    }
}
```

Activar y recargar

```
# Crear symlink para activar el sitio
sudo ln -s /etc/nginx/sites-available/task-manager-jwt \
        /etc/nginx/sites-enabled/task-manager-jwt

# Verificar configuración
sudo nginx -t

# Recargar sin cortar conexiones activas
sudo systemctl reload nginx
```

Mantener el backend activo con PM2

```
npm install -g pm2

cd backend
NODE_ENV=production pm2 start server.js --name task-manager-api

# Persistir al reiniciar el sistema
pm2 startup
pm2 save
```


- Secretos JWT de 64+ caracteres generados aleatoriamente
- DATABASE_URL apuntando a la instancia de BD en la nube
- Cambiar CLIENT_URL al dominio real del frontend

Task Manager JWT

Documentación generada · Abril 2026

Euclides Marín